

Atividade Prática: Sistema PayShield (Processador de Transações)

1. O Enunciado

Você foi contratado para desenvolver o núcleo de um processador de pagamentos. O sistema deve garantir que nenhuma transação seja criada sem os dados obrigatórios e que o saldo de um cartão nunca seja manipulado externamente sem passar pelas regras de negócio.

2. Requisitos da Implementação

Classe A: CartaoCredito (O Molde e o Estado)

Esta classe representa o recurso de dados do cliente.

Atributos (Privados)

numeroCartao (String), titular (String), limiteDisponivel (double) e cvv (String).

Construtor

Deve obrigatoriamente receber o número, titular e o limite inicial. Não deve ser possível criar um cartão “vazio”.

Encapsulamento

- O limiteDisponivel não pode ter um setter direto que aceite qualquer valor.
- Crie um método alterarLimite(double valor) que valide se o novo limite não é negativo.
- Crie getters apenas para o que for necessário (ex: não exponha o CVV por segurança).

Classe B: ProcessadorPagamento (Comportamento e Dependência)

Esta classe é o “motor” que executa as ações.

Gerenciamento de Dependências

O ProcessadorPagamento deve receber uma instância de CartaoCredito no seu construtor para poder operar. Ele depende do cartão para existir.

Métodos (Comportamento)

- `executarVenda(double valor)`: Este método deve verificar se o cartão tem limite. Se sim, subtrai o valor do estado do objeto `CartaoCredito`. Se não, exibe “Transação Negada”.
- `estornarVenda(double valor)`: Devolve o valor ao limite do cartão.

Classe C: Main (Instanciação e Ciclo de Vida)

- Instancie um objeto `CartaoCredito` com R\$ 5.000,00 de limite.
- Instancie o `ProcessadorPagamento` passando o cartão criado (Injeção de Dependência).
- Realize uma venda de R\$ 2.000,00 e mostre o estado do limite após a ação.
- Tente realizar uma venda de R\$ 4.000,00 e demonstre o tratamento de erro.